Ders 9 Genişletilmiş Notu: Mediator Kalıbı, JUnit ile Birim Sınaması ve Kod Kusurlarıyla Refactoring

Ders 9 Genişletilmiş Notu: Mediator Kalıbı, JUnit ile Birim Sınaması ve Kod Kusurlarıyla Refactoring

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Mediator Kalıbı: Amaç ve Font Penceresi Örneği
- 2. Mediator Kalıbının Yapısı, Observer ile Farkı ve Eleştirisi
- 3. Refactoring Nedir, Neden ve Nasıl Yapılır?
- 4. Kod Kusuru (Code Smell) Kavramı
- 5. Birim Sınaması ve Test Odaklı Yaklaşım
- 6. JUnit 3.x ve 4.x
- 7. Kaynak Kitap ve Video Kiralama Örneği
- 8. Duplicated Code ve Extract Method
- 9. Long Method
- 10. Yinelenen Kod İçin Diğer Düzenlemeler: Pull Up Method ve Form Template Method
- 11. Decompose Conditional
- 12. Long Parameter List: Introduce Parameter Object ve Preserve Whole Object
- 13. Replace Temp with Query ve Başarım Tartışması
- 14. Replace Method with Method Object
- 15. Large Class: Extract Class, Extract Subclass, Duplicate Observed Data
- 16. Divergent Change ve Shotgun Surgery
- 17. Primitive Obsession ve Tür Kodu Düzenlemeleri
- Örnek Sorular
- Dersin Toparlaması

1. Mediator Kalıbı: Amaç ve Font Penceresi Örneği

Ders, tasarım kalıpları konusunun son kalıbı olan Mediator ile başlar. Hoca bu kalıbı ayrı bir uygulama örneği yapmadan, yalnız tarif ederek anlatacağını söyler.

1.1 Amaç

"Mediator" arabulucu demektir; kalıpta daha çok "iki haberleşen arasındaki aracı" anlamıyla kullanılır. Elimizde bir nesne kümesi vardır ve bu nesneler kendi aralarında geçici (ad hoc) ya da eşten eşe (peer-to-peer) biçimde etkileşir, haberleşir. Her nesne her nesneyle haberleşirse, n düğümlü tam bağlı bir yapıda yaklaşık n^2 bağlantı oluşur. Her nesne hepsiyle bağlantılı olmasa bile çok sayıda başka nesneyle bağlantılı olur ve ortaya çıkan ilişki karmaşıktır. Soru şudur: bu iletişimi daha derli toplu biçimde yapabilir miyiz? Mediator kalıbı burada devreye girer.

1.2 Örnek: font seçme penceresi

Eski metin editörlerindeki font biçim kutusunu düşünün. Yukarıda bir önizleme (örnek metin) alanı vardır. Bir listeden yazı tipi seçilince yukarıdaki metin kutusuna gelir; kalınlık (medium,

bold), düz ya da italik, punto gibi seçenekler de vardır. Bu bileşenlerin herhangi birinde yapılan her değişiklikte önizleme alanının bundan haberdar olup kendini güncellemesi gerekir.

Hoca eski GUI programlama modellerini hatırlatır: her GUI bileşeninde bir olay gerçekleşince bu olay ilgili forma yansıtılır. Örnekte bu form `FontDialogDirector`'dür; gelen olayın hangi bileşenden geldiğine bakar, gerekirse bileşenleri sorgular ve önizleme alanını düzenler. Java'da bir Listener tanımlanınca da benzer biçimde bu Listener diğer bütün bileşenleri dinleyip önizleme alanını değiştirir.

Alternatif olarak her GUI bileşeni doğrudan önizleme metniyle ilişkili olabilirdi; metnin `setMedium`, `setBold` gibi metotları olsaydı bu mümkündü. Ama amaç genel bir çerçeve kurmaktır: bir GUI bileşeninin kodunu hazırlarken başka bir GUI bileşeniyle bu kadar sıkı bir ilişki kurulamaz. Kurulsa bile bileşenlerin önizleme alanına müdahale kodları arasında epey kopyala-yapıştır benzeri kod oluşur. Hem karmaşıklığı hem kod kopyalamayı önlemek için asıl uygulama mantığı `FontDialogDirector`'e konur; o, kendi içindeki önizleme alanını diğer bileşenleri sorgulayarak istediği gibi düzenler.

1.3 Örnek çözüm (ekrandan, 00:10:00)

Slayttaki örnek çözüm iki madde içerir: GUI nesnelerine ortak bir arayüz hazırlanır ve aralarındaki etkileşimleri genel olarak tanımlayacak başka bir soyut sınıf tasarlanır; bu soyut sınıftan gerçeklemeler oluşturmak için Template Method kalıbı kullanılır. Hoca, "neyin değiştiğini bul ve sarmala" ilkesine göre burada değişen iki farklı olgu bulunduğu için iki farklı sınıf hiyerarşisi oluştuğunu söyler.

```
DialogDirector {ShowDialog(); CreateWidgets() «soyut»; WidgetChanged(Widget) «soyut»}
DialogDirector ◁── FontDialogDirector {CreateWidgets(); WidgetChanged(Widget)}

Widget ──director──► DialogDirector
Widget {Changed()}
   not: Changed() içinde  director->WidgetChanged(this)
Widget ◁── ListBox    {GetSelection()}
Widget ◁── EntryField {SetText()}

FontDialogDirector ──list──►  ListBox
FontDialogDirector ──field──► EntryField
```

`EntryField`, `ListBox` gibi bileşenler `Widget`'ın alt sınıflarıdır. Bir widget'ta durum değişikliği olduğunda bunu Mediator'a bildirmesi gerekir; bu yüzden `Changed()` olayı vardır. `Changed()` içinde widget, hangi director ile eşleştiyse ona "ben değiştim" der.

2. Mediator Kalıbının Yapısı, Observer ile Farkı ve Eleştirisi

2.1 Observer'a benzerlik ve fark

Hoca şemanın yalnız widget'tan director'e bildirim kısmına bakıldığında Observer gibi düşünülebileceğini, hatta yukarıya bir soyut Observer sınıfı, altına da somut Observer örnekleri eklenmiş gibi görülebileceğini söyler. Ama buradaki amaç değişikliklerden haberdar etmek değildir. Sınıf şeması çok benzese de amaç, farklı bileşenler arasında normalde eşten eşe kurulacak $m \times n$ ilişkiyi merkezi bir noktada toplamaktır.

Her topolojinin avantajı ve dezavantajı vardır. Merkezi topolojide merkez düğüm çökerse bileşenler kendi aralarında haberleşemez. Öbür türlü ise değişik türdeki bileşenlerin haberleşmesi için istekçilere karmaşık kod düşer. Probleme hangisi daha uygunsa o seçilir. Mediator kalıbında Observer'dan farklı olarak tek bir Mediator nesnesi vardır; birçok eş arasındaki ilişki eşten eşe değil, eşten Mediator'a ve Mediator'dan ilgili diğer eşe gidecek biçimde kurulur. Hocaya göre Mediator ile Observer arasındaki fark budur.

Hoca font penceresi örneğinde hangi kalıbı kullanacağımız sorulursa doğru yanıtın Mediator olacağını, çünkü bu örneğe "nokta atışı" hitap eden kalıbın bu olduğunu söyler.

2.2 Kalıp yapısı (ekrandan, 00:12:30)

Slayt: “Colleague gerçeklemeleri kendi aralarında doğrudan değil, bir Mediator gerçeklemesi üzerinden yaparlar.”

```
Colleague ──mediator──► Mediator
Mediator  ◁── ConcreteMediator
Colleague ◁── ConcreteColleague1
Colleague ◁── ConcreteColleague2
ConcreteMediator ──► ConcreteColleague1
ConcreteMediator ──► ConcreteColleague2
```

Soyut bir `Colleague` (meslektaş ya da eş) sınıfı vardır; somut alt sınıfları olur. Somut meslektaşlar, üst sınıftan aldıkları `Mediator` referansı üzerinden diğer eşleriyle haberleşir; bu yüzden `Colleague`’dan `Mediator`’a birebir sahiplik ilişkisi gerekir. Somut Mediator oluşturulunca somut meslektaşlar birbirlerine mesaj gönderiyor gibi olur, ama aslında mesajı `ConcreteMediator`’a gönderirler. Bu yüzden `ConcreteMediator`’dan da somut meslektaşlara birebir sahiplik bağlantıları gerekir: `ConcreteColleague1` nesnesinden çıkan mesaj doğrudan `ConcreteColleague2` nesnesine gitmez; önce Mediator’a gider, sonra Mediator ilgili işi `ConcreteColleague2` türündeki nesneye yaptırır.

Duruma göre her mesajı her eşe göndermemek de seçilebilir. Belirli meslektaş türleri arasındaki iletişimler için farklı somut Mediator alt sınıfları da olabilir; kardeş sınıflarla değişik çeşitlemeler kurulabilir.

Nesne ilişkisi olarak düşünüldüğünde bütün meslektaşlar birbirleriyle değil Mediator ile ilişkilidir. Hoca ekranda şunu gösterir: Mediator’ın bir meslektaşa referansı yoksa, o meslektaşa mesaj gidemez. Örneğin A meslektaşı Mediator üzerinden B’ye mesaj gönderebilir ama Mediator A’yı bilmiyorsa B’den A’ya mesaj gidemez. Mediator tüm meslektaşlara işaretçi tutarsa herkes birbirine istediği mesajı somut Mediator üzerinden gönderebilir.

2.3 Kalıbın eleştirisi

Hoca kalıbı şöyle eleştirir: karmaşıklık meslektaşlardan Mediator’a taşınmış olur. Ortadaki karmaşıklık belki biraz daha kolay yönetilebilir hâle gelmiştir ama yine de karmaşıktır; bu yüzden kalıp çok fazla şey vaat etmez.

- Meslektaşlar arasındaki mesajlaşma protokolleri ve aralarındaki işler iyi tanımlanmışsa ve zamanla sık değişmeyecekse, karmaşıklığı bir nebze azaltmak için Mediator kalıbı kurulabilir. Karmaşıklık sıfıra inmez, çok da azalmaz.
- Etkileşimler iyi tanımlanmamışsa ve zaman içinde sürekli değişebilecekse Mediator kalıbını hiç kurmamak daha iyidir.

Hoca, karmaşık bir kalıp olmadığı için ayrı bir örnek çözüm yapmadan tasarım kalıpları konusunu burada bitirir.

3. Refactoring Nedir, Neden ve Nasıl Yapılır?

Dersin ikinci büyük konusu kod kusurları (code smells) ve refactoring’dir. Hoca bu konuda öğrencilerin soru ve görüşlerini özellikle beklediğini, ilerledikçe tartışmalı konulara geleceklerini söyler.

3.1 Tanım (ekrandan, 00:20:00)

Slayttaki tanımlar:

- Bir kod parçasının ne yaptığını değiştirmeden nasıl yaptığını değiştirmeye refactoring adı verilmiştir.
- Yazılımın dışarıdan görünen davranışı değiştirilmeden iç yapısının değiştirilmesi anlamındadır.

Hoca, refactoring’in kod kusurlarından bağımsız bir kavram olduğunu belirtir. Dışarıya açık sınıfların ve metotların verdiği hizmetler, metot imzaları, istemcilerin eriştiği arayüzler bozul-

madan iç yapı değiştirilir. Bunun tersine mühendislikle (bilinmeyen bir kaynak kodunu ya da sınıfı çözmeye çalışmak) karıştırılmaması gerekir.

3.2 Amaç

Slayta göre amaç, mevcut kod üzerinde kodun iç kalite özelliklerini daha iyiye götürecek değişiklikler yapmaktır:

- Kodun anlaşılabilirliğinin artırılması
- Koda esneklik kazandırılması
- Yeniden kullanılabilirliğin artırılması
- Yüksek uyum ve düşük bağımlılığın sağlanması

Neden iç yapıyla uğraşırız? Belki ilk başta hızlı ve kirli (quick and dirty) bir çözüm yapmışızdır; gereksinimler değiştikçe "bu tasarımla daha fazla ilerleyemeyeceğiz" diye karar verebiliriz. Kodda değişiklik yapma ihtiyacı kaçınılmazdır; o iş de eklensin, bu iş de eklensin derken yazılım yamalı bohçaya döner ve sonunda bütünüyle yeniden tasarım gereken noktaya gelinir. Dönemin başında konuşulan iyi tasarım ölçütlerine (gereksinim değişikliklerine dayanıklılık ve esneklik) ek olarak, yazılımın bakım masraflarını azaltan her değişiklik refactoring kapsamında düşünülür. Bakımı zorlaştıran her konu bir kusur sayılır, özel adlar verilir ve düzeltilir. İç kalite özellikleri cinsinden bu, uyumu yükseltmek ve bağlaşımı düşürmek demektir.

3.3 Küçük adımlar

Önerilen yol refactoring'i küçük adımlarla, artımsal olarak, çevik yaklaşımlarla yapmaktır. Alternatif, örneğin 2.0 sürümünden 2.2'ye geçerken bütünüyle büyük bir yeniden tasarım ya da yepyeni bir proje yapmaktır; bu altı ay, bir yıl sürebilir. Bu sürede sahada yeni değişiklikler çıkar ve onları da eklemeye kalkınca 2.2 sürümünün de yeterince esnek olmadığı anlaşılabilir. Küçük dokunuşlarla düzeltmek daha kolay olduğu için refactoring eylemleri küçük adımlarla yapılır.

3.4 Sınamayı ihmal etmemek

Yapılan değişiklik ilk başta düzgün çalışıyor gibi görünse de bütün birim, tümleştirme ve hatta sistem testleri yeniden çalıştırılmadan sistemin başka bir yerinin bozulmadığından emin olunamaz. Bu yüzden her değişiklikten sonra yeni bir sınama yapılmalıdır. Her seferinde bütün testleri çalıştırmak gerekmeyebilir: değişikliğin etkilediği varlıkları ve onların bir adım ötesini kapsayan senaryolar çalıştırılır; iki üç haftalık bir artımın sonunda daha kapsamlı testler yapılabilir.

4. Kod Kusuru (Code Smell) Kavramı

Refactoring, kod "kötü kokmaya" başladığında yapılır. İngilizcede "something is smelling" denince genellikle kötü bir şey kastedilir. Kodun kötü kokması, yamalı bohçaya dönmesi ve gereken yeni değişiklikleri kaldıracak esnekliği kaybetmeye başlaması demektir. Buna "kötü kokan kod" da denebilir, ama hoca "kusurlu kod" ya da "kod kusuru" demeyi tercih eder.

> **Hocanın vurgusu:** Kod kusuru hata (bug) değildir. Kesinlikle hata değildir; kusurdur. Kodun bakımını zorlaştıran bir tasarım sorunudur.

Eğitimli bir göz ya da deneyimli bir programcı değilsek yazılımın yamalı bohçaya dönmeye başladığını fark edemeyebiliriz; deneyimli olsak bile günlük koşturmacada gözden kaçabilir. Kötüleşme sinyallerini erken alabilirsek onları daha çabuk ve kolay refactor edebiliriz. Bunun için kusurların neler olduğunu bilmek gerekir; aksi hâlde kusurlar birikir ve kolay düzeltilebilecek nokta geçilir.

Küçük adımlarla yapılan bir iyileştirme sistemin bir parçasını düzeltirken tasarımın geri kalanında yeni bir uygunsuzluk, yeni bir kusur ortaya çıkarabilir. Bu yüzden refactoring ve birim sınaması birbirini izleyen bir sıra ya da sarmal biçiminde yürütülür.

5. Birim Sınaması ve Test Odaklı Yaklaşım

5.1 Birim sınaması (ekrandan, 00:30:00)

Slayttaki bilgiler:

- Birim sınaması en küçük yazılım bileşeninin sınanmasıdır; nesneye dayalı programlamada bireysel sınıfların sınanmasıdır.
- Kodlamadan önce (TDD), kodlama sırasında ya da kodlamanın ardından tasarlanabilir.
- Bir sınıfın tek başına yürütemediği sorumlulukların sınanması için bu sınıfın ihtiyaç duyduğu diğer sınıfların yerine geçecek kod gerekebilir: vekil, sahte, yalancı kod ya da sınıf (stub, dummy, surrogate, proxy). Vekil, yalnız ihtiyaç duyulan sınıflar gerçeklenene dek kullanılır.

5.2 Test odaklı tasarım

Hoca test driven design (TDD) yaklaşımını anlatır: kimileri tek satır iş mantığı yazmadan önce birim testlerini yazmaya başlar. İyi bir analiz yapılmışsa üye alanlar, üye metotlar ve sınıflar arası ilişkiler bellidir. Gelişmiş bir yazılım geliştirme aracı sınıf şemasından kabataslak iskelet kod üretebilir; bu noktada derleme hatası almadan birim testleri hazırlanabilir. Metot gövdeleri boş olduğu için testler ilk başta başarısız olur, ama henüz kodlanmamış bir sınıfın testinin başarısız olması sorun değildir; o testler askıya alınıp diğerleriyle ilerlenir.

Vekil sınıflar da bu süreçte kullanılır: iskelet kod yerelde tutulur ve kodlaması bitmemiş sınıfların yerine geçer. Varsayım, vekil sınıfların işini her zaman doğru yaptığıdır; örneğin bir metot her zaman `return true` döndürür, işlem başarılı olmuş gibi. Böylece o sınıfa ihtiyaç duyan diğer kod test edilir. Diğer sınıfın gerçek kodu tamamlanınca merkezi depodan o sürüm alınır ve tümleştirme testleri başlar. Hocaya göre test yapmamanın mazereti yoktur.

> **Hocanın vurgusu:** Bir şeyi doğru test edemiyorsanız ya da test etmekte zorlanıyorsanız büyük ihtimalle doğru tasarlayamamışsınızdır ya da orada bir hata vardır.

Herkes testle başlamak zorunda değildir; bir kısım kodlama yapıp ardından test durumları hazırlanabilir. Ama bütün kod bittikten sonra test yazmak geç kalmaktır. Hocanın önerisi "biraz kod, biraz test"tir; örneğin bir sınıfın kodunun %50'si tamamlanmadan test yazmamak da bir tercihtir. Önemli olan test yapmaktır. Hoca sektörde sınama etkinliklerine toplam projenin %10'u kadar zaman ayrıldığını gördüğünü ve sınamaya yeterli özen gösterilmediğini söyler.

6. JUnit 3.x ve 4.x

6.1 JUnit hakkında (ekrandan, 00:30:00)

Sınamaların hızlı ve verimli yapılabilmesi önemlidir. JUnit, Java ile yazılmış kodun birim sınamaları için bir çerçeve programdır (framework). xUnit ailesinin bir üyesidir; diğer diller için de sürümleri vardır (C# için csUnit). Açık kaynaktır. Eclipse'te IDE ile hazır gelir (build path → add libraries), NetBeans'te de hazır gelir. Hoca dersin sonunda üç sınıflık basit bir tasarım üzerinde adım adım bütün bir refactoring yapılacağını ve bu sırada birim testlerinin çalıştırılacağını söyler.

JUnit'in ilk tanıdığı sürümü 3.x'tir; 4.x ile annotation desteği gelmiştir. Geriye doğru uyumluluk korunur: 3.x'te yazılan testler 4.x'te, 4.x'te yazılanlar sonraki sürümlerde çalışır.

6.2 JUnit 3.x yapısı

Uygulama mantığını oluşturan sınıflarımız `AppClass1`, `AppClass2` gibi olsun. Test sınıfları aynı pakette ya da dışarıda bir pakette tutulabilir; dışarıda tutmak daha mantıklıdır. En azından paketler jar dosyasına dönüştürülürken test sınıfları içine konmaz.

- Projeye JUnit çerçevesi dahil edilir. Test yazarken `TestCase` sınıfı ve onun bağımlı olduğu `Assert` sınıfının `assertXxx` metotları kullanılır.
- Her uygulama sınıfı için bir test sınıfı yazılır; adının `Test` ile başlaması 3.x'te zorunludur. Örneğin `AppClass1` için `TestAppClass1`. Aralarında sahiplik ilişkisi gerekmez, en azından kullanma ilişkisi olur.

- Test sınıfına `testXxx` metotları eklenir; her biri bir test durumudur (test case). Metot adları `test` ile başlamak zorundadır.
- İstenen test sınıfları bir suite içinde birleştirilip tek adımda çalıştırılabilir; farklı gruplar için farklı suite'ler kurulabilir. Suite de adı `Test` ile başlayan bir sınıfta, geriye bir `Test` nesnesi döndüren `suite` metoduyla oluşturulur. Suite kurucusuna bir ad ve test sınıflarının adları verilir.
- Test sınıfları `TestCase`'ten kalıtımla türetilir ve `setUp` metodunu kalıtımla alır. `setUp`'a yazılan kod "sahneyi hazırlar": nesneler oluşturulur, ilişkiler kurulur, set metotlarıyla test ortamı hazırlanır. Her `testXxx` metodu çalışmadan önce `setUp` otomatik olarak çalıştırılır.
- Her test metodunun sonunda bir `assert` metoduyla sonuç denetlenir.

Kara kutu mantığıyla çalışılır: verilen girdi bilinir (`setUp` ve test metodunda verilen), doğru kodlanmışsa elde edilmesi beklenen bir değer vardır ve sistemin gerçekte ürettiği sonuç elde edilir. Beklenen ile elde edilen aynıysa test başarılıdır.

- `assertEquals(beklenen, sonuç)`: iki nesne eşitse test geçer.
- `assertNotEquals`: testin başarılı olması için iki nesnenin eşit olmaması gerekiyorsa.
- `assertTrue` / `assertFalse`: bir boolean ifadenin doğru ya da yanlış olması gerekiyorsa.
- `assertNull` / `assertNotNull`: sonucun `null` olması ya da olmaması gerekiyorsa.

Bu metotların hepsi statiktir.

6.3 JUnit 4.x gelişmeleri (ekrandan, 00:40:00)

Slayttaki bilgiler: geriye doğru uyumluluk korunmakla birlikte JUnit 4 ile annotation desteği gelmiştir (büyük/küçük harf duyarlılığı vardır). Artık sınama sınıflarının `TestCase` sınıfından kalıtımla türetilmesi gerekmez, ancak şu eklemeler yapılmalıdır:

```
import static org.junit.Assert.*;
import org.junit.*;
```

Artık sınama metotlarının adlarını `test` kelimesiyle başlatmak gerekmez; ilgili metotların başına `@Test` koymak yeterlidir. `setUp` adlı metodun yerine `@Before` annotation'ı gelmiştir:

```
@Before
public void setUp() { /*Preparations*/ }
@Test
public void testSomething() { /*Do test*/ }
```

Hoca `setUp` adını yalnız anlaşılsın diye koyduğunu, metoda "haydiHazırlan" gibi herhangi bir ad verilebileceğini, yeter ki başına `@Before` konduğunu söyler. Test metotları da "denemeYap" gibi adlandırılabilir.

Exception tanımlanan yazılımlarda, atılması gereken exception'ların gerçekten ortaya çıkıp çıkmadığının sınanması da mümkün olmuştur:

```
@Test(expected=SomeException.class)
public void testTheException() throws SomeException {
    doSomethingThatCreatesTheException();
}
```

Checked exception'lar dilin gereği olarak `throws` ile bildirilmek zorundadır; unchecked olanlar zorunda değildir. Bu testte exception oluşmuşsa test başarılı, oluşmamışsa başarısızdır. Bu özellik olmadan exception oluşunca program akışı tamamen kesilirdi. Oysa amaç bunu otomatik olarak test etmek ve test başarılı da olsa başarısız da olsa diğer testlerin sürmesidir; 3.x'te bu yapılamıyordu, 4.x'te yapılabiliyor.

7. Kaynak Kitap ve Video Kiralama Örneği

7.1 Kaynak ve anlatım yöntemi

Refactoring ve kod kusurları konusunun kaynağı Martin Fowler'ın *Refactoring* kitabıdır; dersin içeriği oradan alınmıştır. Hoca kitaptaki kod kusurları ve refactoring eylemlerinin yaklaşık yarısını, dersin kalan haftalarına sığacak biçimde seçmiştir. Kitap, kusurları önce tanıtmadan doğrudan bir refactoring örneğine dalar ("denize atlayarak yüzme öğrenmek" gibi). Hocanın

tarzı ise önce konuyu anlatıp sonra örneği uygulamaktır: sırayla bir kod kusuru tanıtılır, neden sakıncalı olduğu söylenir, onu düzeltecek refactoring eylemi anlatılır.

> **Hocanın vurgusu:** Her kod kusuru ve her refactoring eylemi kitaptaki İngilizce özgün adıyla anılır; sınavlarda da bu özgün adların kullanılması beklenir.

Hoca anlatımında önce bir kod kusurunu, ardından onunla ilgili ilk refactoring'i, sonra varsa aynı kusurla ilgili diğer refactoring'leri anlatır ve sonraki kusura geçer. Sonraki bir kusur daha önce anlatılmış bir refactoring ile de çözülebiliyorsa yalnız adıyla ona gönderme yapar. Ders notlarında da kitaptaki gibi kodun önceki ve sonraki hâli (ya da sınıf şeması) yan yana verilir.

7.2 Video kiralama senaryosu

Örneğin tamamı dönemin sonunda yapılacak, ama kusurların çoğu bu senaryo üzerinden örnekleneceği için ana yapı şimdiden tanıtılır. Senaryo basit bir video kaset kiralama uygulamasıdır.

- `Customer` sınıfının `statement` metodu bir makbuz üretir: müşterinin aldığı video kasetlerin adları, türleri, kaç gün kiralandığı, her kaset için borç, kiralamadan kazanılan bonus puanı, toplam borç ve toplam bonus.
- `statement` ekrana yazmaz, `String` döndürür. Doğrudan ekrana yazsa test etmek zor olurdu; `String` dönünce beklenen ifadeyle karşılaştırılarak doğru mu yanlış mı kararı rahatça verilir.
- Bir `Customer` sıfır ya da daha çok kiralama işlemi yapar; bunlar `Rental` sınıfıyla tutulur (`daysRented` üyesi vardır).
- Her kira kaydı bir `Movie` (aslında video kaset) ile ilişkilidir. Her filmin bir fiyat kodu vardır, çünkü üç tür film vardır: yeni çıkan filmler, normal filmler ve çocuk filmleri. Türlere göre farklı ücret ve puan kazanma durumları vardır.
- `statement` bütün kira kayıtlarını dolaşır: kaydın hangi filme ait olduğunu, filmin fiyat kodunu ve kaç gün kiralandığını öğrenir, ücreti hesaplayıp kümülatif toplama ekler, benzer biçimde bonusu hesaplar ve sonunda biriktirdiği bilgileri `String` olarak döndürür.

Hoca bu şemanın kitaptan alındığını, yalnız sıra şemasını (sequence diagram) kendisinin çizdiğini söyler. Geri dönüş oklarını çizmeyenler çok olsa da hoca onları çizmekten yanadır, çünkü bir çağrının iç içe mi yoksa sıralı mı olduğu ancak böyle iyi görülür.

7.3 Eclipse'te ilk kod ve testler (ekrandan, 00:50:00)

Hoca Eclipse'te refactoring adımlarını `fowler_00`, `fowler_01`, … `fowler_10_alt`, `fowler_11_alt` paketlerinde, testleri ise `testing` klasöründe aynı paket adlarıyla tutar. `fowler_00` henüz hiç refactor edilmemiş ilk hâldir. Test sınıfları farklı dizinde olsa da aynı pakettedir; Eclipse'te bu yapılandırılabilir ve böylece test kodunu aynı pakette tutmanın sakıncası azalır, yalnız üretim kodunun dizini paketlenir. Ekranda görünen ilk kodun başı:

```
package fowler_00;

import java.util.*;

public class Customer {
    private String _name;
    private Vector<Rental> _rentals = new Vector<Rental>();
    public Customer (String name){
        _name = name;
    }
    public void addRental(Rental arg) {
        _rentals.addElement(arg);
    }
    public String getName (){
        return _name;
    }
    public String statement( ) {
        double totalAmount = 0;
```

```
        int frequentRenterPoints = 0;
        Enumeration<Rental> rentals = _rentals.elements();
        String result = "Rental Record for " + getName() + "\n";
        // ... (ekranda devamı görünmüyor)
```

`TestCustomer` sınıfı `TestCase`'ten türetilir; bir müşteri ve farklı türden filmler tanımlanır. `setUp`'ta müşteri ve filmler ilklendirilir ama aralarındaki ilişkiler kurulmaz, çünkü bir test hiç kiralama yapılmamışken basılan makbuzun doğruluğunu sınar. Diğer test durumları şunlardır: her türden birer filmin uzun süre kiralanması (borç ve bonus doğru mu), kısa süre kiralanması, yeni çıkan filmlerde fazladan bonus kazanılması. Her testte ilişkiler kurulur, beklenen makbuz metni oluşturulur ve `statement` sonucuyla karşılaştırılır. Çalıştırınca beş test durumunun hepsi geçer ve yeşil çubuk görülür; biri başarısız olsaydı ilgili yerde kırmızı yazılır ve bütün çubuk kırmızı olurdu.

Yalnız fiyat kodu, iki üye alan, kurucu ve get/set metotları olan `Movie` sınıfı bile test edilir: `TestMovie`'de üç farklı türden film oluşturulup farklı türde oldukları `assert` metotlarıyla denetlenir. Basit olduğu için ikili karşılaştırmalar (1-2, 1-3, 2-3) üç ayrı test durumu yerine tek test durumunda toplanmıştır.

7.4 Üye alan adlandırma ve suite (ekrandan, 01:10:00)

Sınıf şemasında ve kodlarda üye alanların adları alt çizgiyle başlar. Slayt: "Kod örneklerinde alt çizgi ile başlayan değişkenler, nesnenin üye alanlarıdır. IDE'lerde ise üye alanlar farklı renk ile vurgulanır." Bu C++'tan kalma bir alışkanlık gibi görünse de asıl önemli neden şudur: IDE'de renklendirmeyle neyin üye alan, neyin yerel değişken, neyin parametre olduğu hemen anlaşılır; ama kitap ve notlar çoğu zaman kodun yalnız bir parçasını verir. Alt çizgiyle başlayan bir şey görülürse o mutlaka üye alandır.

JUnit 3 ile suite kurmak için `suite` metodunun içinde suite kurucusu bir adla çağrılır ve test durumlarını içeren sınıfların adları verilir. Hoca `TestAll` suite'ini çalıştırır: bir sınıfta bir, diğerinde beş olmak üzere toplam altı test durumu çalışır ve hepsi başarılıdır.

8. Duplicated Code ve Extract Method

8.1 Yinelenen kod (ekrandan, 01:10:00)

Slayttaki tanım: yinelenen kod, aynı kodun veya çok benzer kod yapısının birden fazla yerde tekrarlanması durumudur. Bu kodun işlevselliğinde değişiklik gerektiğinde kodun tekrarlandığı her yerde aynı işlemlerin tekrarlanması gerekecektir. Ortaya çıkan sonuç bir bakım kabusudur.

Hoca, kopyalanan kod uzun ve karmaşıksa ve birebir aynı değil de ufak varyasyonlarla yapıştırılmışsa, ilk anda günün kurtarıldığını ama değişiklik geldiğinde düzeltmenin çok daha pahalıya patlayacağını söyler. Bu kusur mutlaka yok edilmelidir. Çözüme götüren ilk düzenleme Extract Method'dur; yinelenen kodu toplamaya yönelik diğer düzenlemelere ileride değinilir.

8.2 Extract Method

"Extract", bir şeyin özünü çıkarmak gibi belli bir kod parçasını bağımsız hâle getirmek anlamındadır. Birçok yerde kopyalanan kod çıkarılıp tek bir metot altında toplanır. Extract Method birçok kod kusurunu düzeltmek için kullanılabilen temel bir refactoring eylemidir.

> **Hocanın vurgusu:** Yeni oluşturulan metoda çok düzgün bir ad verilmelidir. Metodun ne işe yaradığı adından net anlaşılmalıdır; ayrıntılar gövdeye bakınca anlaşılır. Ad çok uzun olabilir, önemli değildir; yeter ki yaptığı işi anlatsın ve koda yorum koyma ihtiyacı bile duyulmasın.

İlk örnekte bir `printOwing` metodu önce bir banner basar, sonra yorumla "ayrıntıları bas" denen iki satırla ayrıntıları yazar. Burada yinelenen kod yoktur, ama metot aslında iki ayrı iş yapar ve ikinci işin ne olduğunu açıklamak için yorum gerekmiştir. Bu iki satır kesilir, yorumla aynı adı taşıyan yeni bir `printDetails` metoduna yapıştırılır ve `printOwing` içinde bu metot (borç miktarı parametre verilerek) çağrılır. Sonuçta `printOwing`'in ne yaptığını anlamak için

altı satır yerine iki satır okunur: banner basılıyor, borç miktarıyla ilgili ayrıntılar basılıyor. Ayrıntı gerekirse o metodun içine girilir. Bu, büyük problemleri daha küçük ve kolay yönetilebilir parçalara ayıran “böl, parçala, yönet” ilkesinin uygulamasıdır.

9. Long Method

9.1 Kusur

Long Method aşırı uzun metottur. Metot uzadıkça anlaşılması zorlaşır, hata olasılığı artar, gereksinim değişikliklerinden etkilenme olasılığı artar ve uyum düşer. Uzun metot daha çok iş yapıyor demektir ve bu işler birbiriyle uyumlu olmayabilir: dört iş varsa, birincisi ikincisiyle, ikincisi üçüncüsüyle biraz uyumlu olabilir ama birinci ile dördüncü arasındaki uyum çok daha düşüktür. Uyumu yükseltmek, anlaşılabilirliği artırmak ve nesneye yönelimin “böl, parçala, yönet” ilkesine uymak için aşırı uzun kod bölünmelidir. Bunun için Extract Method yapılır: uzun metotta hangi alt işler yapılıyorsa her biri ayrı bir metot olarak çıkarılır.

9.2 `printOwing` örneği (ekrandan, 01:20:00)

```
/*** ÖNCE: ***/
void printOwing( ) {
  Enumeration e = _orders.elements();
  double outstanding = 0.0;

  // print banner
  System.out.println("******************");
  System.out.println("* Customer Owes *");
  System.out.println("******************");

  // calculate outstanding
  while (e.hasMoreElements()) {
    Order each = (Order) e.nextElement();
    outstanding += each.getAmount();
  }

  //print details
  System.out.println ("name:" + _name);
  System.out.println ("amount" +
     outstanding);
}
```

```
/*** SONRA: ***/
void printOwing( ) {
  printBanner();
  printDetails(getOutstanding());
}

void printBanner() {
  System.out.println("******************");
  System.out.println("* Customer Owes *");
  System.out.println("******************");
}

double getOutstanding() {
  double outstanding = 0.0;
  for (Order each : _orders)
    outstanding += each.getAmount();
  /* for-each ile döngü kısaldı! */
  return outstanding;
}
```

```
void printDetails (double outstanding) {
  System.out.println ("name:" + _name);
  System.out.println ("amount" +
     outstanding);
}
```

Önceki metotta `_orders` üye alanının (alt çizgi) elemanları bir `Enumeration` ile dolaşılır, kümülatif toplam değişkeni tanımlanır, banner yazılır, borç hesaplanır ve en sonda müşterinin adı ve miktar yazılır. Üç ayrı iş vardır: banner yazmak, borcu hesaplamak, ayrıntıları vermek. "Bir metot bir iş yapmalı" ilkesiyle `printBanner`, `getOutstanding` ve `printDetails` ayrı ayrı çıkarılır. Ayrıntıların toplam borcu bilerek yazılması gerektiği için iç içe kullanım yapılır: önce `getOutstanding` hesaplanır, sonuç doğrudan `printDetails`'e verilir.

Hoca burada bir geçici değişken kullanmadığını vurgular. `double outstanding = getOutstanding();` yazıp sonra parametre olarak verseydi üç satır olurdu, şimdi iki satırdır. Amaç kodu kısaltma takıntısı değil, boş yere geçici değişken kullanmamaktır. Geçici değişkenler tanımlandıkça kodun uzama olasılığı artar: o sonuç şurada da burada da kullanılır, bir geçici değişken bir başkasını izler; kod hem uzar hem karmaşıklaşır. `getOutstanding` içindeki `outstanding` ise kümülatif toplam olduğu için kaldırılamaz; ama en azından `while` döngüsü yerine for-each kullanılarak kod daha kısa hâle getirilmiştir.

Soldaki de sağdaki de anlaşılır, ama sağ tarafta her metot daha odaklanmış bir iş yapar; anlaşılması daha kolaydır, uyumu daha yüksektir ve adlar öyle seçilmiştir ki yorumlara gerek kalmamıştır. Odaklanmış parçaların yeniden kullanılabilirliği de daha yüksektir. Hoca "yorum yazmayın" demediğini, ama kod kendini açıkça anlatıyorsa yoruma gerek kalmadığını belirtir.

9.3 Parça sayısı ve başarım tartışması

"Böl, parçala, yönet" mantığının bir sakıncası, yönetilmesi gereken parça sayısının artmasıdır. Bu bir zarardır, ama genel bir ödünleşimdir (trade-off) ve kazanılan yarar karşısında küçük kalır. IDE bir metottan diğerine kolayca atlamayı sağlar; metotlar kendini iyi anlattığı için üst düzey bilgi üst metottan alınır, ayrıntı gerekiyorsa ilgili metoda tıklanır.

Çalışma anı başarımı üzerinden itiraz edilirse hocaya göre bu itiraz haklı değildir. C++'ın ilk zamanlarında bir metottan diğerine geçişteki bağlam değiştirme (context switching) işlemleri daha masraflıydı. Modern derleyiciler daha akıllıdır, daha optimize kod üretir ve çalışma zamanına düşünülenden çok daha az ek yük getirir. Küçük nesne ve metotlar bellek ayak izi (memory footprint) ya da çalışma anı başarımı açısından hissedilebilir bir ek yük getirmez.

Tartışma şu eksende olmalıdır: yapılan refactoring eylemi çalışma anı başarımına ne kadar ek yük getirdi ve bu önemli mi? Önemliyse o refactoring yapılmaz, diğer kısımlar refactor edilir. Sistemin en zayıf noktası darboğazın oluştuğu yerdir; büyük olasılıkla bu bir ağ haberleşmesidir. Uzun bir metottan metot çıkarmak insanın hissedemeyeceği bir ek yük getirir; asıl optimize edilmesi gereken darboğazın yanında hiç kalır. Gerçekten dikkatle optimize edilmiş bir kodu refactor etmeye çalışılıyor ve başarım önemli ölçüde düşüyorsa, adımlar zaten küçük olduğu için sistem testleri çalıştırılır ve başarım çok düştüyse adım hemen geri alınır.

10. Yinelenen Kod İçin Diğer Düzenlemeler: Pull Up Method ve Form Template Method

10.1 Pull up ve pull down

Pull up ya da pull down gibi adlar görülünce bir kalıtım hiyerarşisi söz konusudur: pull up üst sınıfa, pull down alt sınıfa taşımaktır. Çıkarılan metot aynı sınıfa konuyorsa yapılan eylem yalnız Extract Method'dur. Kalıtım hiyerarşisinde başka bir sınıfa konuyorsa, üst sınıfa taşımak Pull Up, alt sınıfa taşımak Pull Down olarak adlandırılır.

10.2 Pull Up Method (ekrandan, 01:30:00)

Slayt: "Kardeş sınıflarda yinelenen metotlar varsa, bunlar üst sınıfa çekilerek bir tek yerde kodlanır."

```
ÖNCE:                                SONRA:
Employee                             Employee {getName}
Employee ◁── Salesman {getName}      Employee ◁── Salesman
Employee ◁── Engineer {getName}      Employee ◁── Engineer
```

`Employee` sınıfından kalıtımla türetilen `Salesman` ve `Engineer` sınıflarının ikisinde de ayrı bir `getName` metodu tanımlanmıştır. Aynı şey olduğu için metot üst sınıfa çekilir ve alt sınıflara kalıtımla aktarılır. Arada ufak farklar olacaksa alt sınıf metodu `override` eder; belki tamamen ezmek yerine önce `super` ile üst sınıftaki hâlini çağırıp üstüne bir şey ekler.

10.3 Form Template Method

Yinelenen kod kardeş sınıflarda birebir aynı değilse (kısmi yinelenen kod) Pull Up doğru olmaz, çünkü üst sınıfa çekilen metot ona ihtiyaç duymayan alt sınıflara da aktarılabilir. O zaman Form Template Method refactoring eylemi yapılır; yani Template Method tasarım kalıbı kullanılır. Template Method sayesinde gereksiz kısımların kalıtımla aktarılması engellenir.

Örnekte bir `Customer` sınıfı (son hesap kesim tarihi ve faturaya kalem ekleme metodu olan) ve alt sınıfları `RegularCustomer` (normal müşteri) ve `PreferredCustomer` (tercihli müşteri) vardır. İkisinin de belirli başlangıç ve bitiş tarihleri arasındaki masrafı hesaplayan bir `chargeFor` metodu vardır. Metot imzaları aynıdır, ama tercihli müşteri için çok farklı bir hesaplama yöntemi kullanılır. Örneğin metotlardan biri 15, diğeri 20 satırsa bunların 7-8 satırı ortak, kalan 3-5 satırı farklıdır. Hiçbir şey yapılamıyorsa en azından metot üst sınıfa soyut olarak konur ve gövdeler alt sınıflarda olduğu gibi bırakılır. Yapılabiliyorsa Template Method kalıbı kullanılır. Bunun sonucunda alt sınıfların birinde boş gövdeli bir metot kalabilir, ama yine de önceki hâlden daha iyidir ve bakımı daha kolaydır.

10.4 İlgisiz sınıflarda yinelenen kod

Yinelenen kod birbiriyle tamamen alakasız ya da birkaç adımlık zincirleme ilişkiyle bağlı sınıflarda ortaya çıkmışsa, bu analiz aşamasında sorumlulukların hiç doğru dağıtılmadığı anlamına gelir. Ama büyük bir tasarım değişikliği yapacak durumda olmayabiliriz; küçük refactoring eylemleriyle devam etmek isteriz. O zaman yinelenen kod bulunduğu sınıflardan çıkarılıp uygun bir yere konur ve diğer sınıflar oradan çağırır. Aralarında en azından bir sahiplik ya da kullanma ilişkisi büyük olasılıkla vardır; belki ters yönde yoktur. Bağlaşım bir adım yükselir; hoca bunu “razı olmak” değil “sineye çekmek” olarak niteler. Yüzlerce sınıf ve ilişkiden oluşan karmaşık bir sistemde 100 ilişkinin 105’e çıkması büyük fark yaratmaz; birçok yere dağılmış yinelenen kod ise çok daha kötü bir bakım kabusudur.

Özet: çıkarılan metot aynı sınıfta kalıyorsa Extract Method, kalıtım hiyerarşisinde üst sınıfa geçiyorsa Pull Up Method, doğrudan üst sınıfa çekmek uygun değilse Form Template Method yapılır.

11. Decompose Conditional

Long Method’u çözüme götüren diğer düzenlemeler Decompose Conditional, Replace Temp with Query ve Replace Method with Method Object’tir. Decompose Conditional, metodun uzamasının ana nedeni bir if-else-if zinciri, switch-case ya da iç içe döngüler gibi karmaşık karar verme kısımlarıysa uygulanır ve o kısmı basitleştirmeye odaklanır. Geçici değişkenler metotları uzatıyorsa Replace Temp with Query uygulanır. Bu ikisi de yetmezse daha ağır ve karmaşık bir düzenleme olan Replace Method with Method Object yapılır.

11.1 Örnek (ekrandan, 01:40:00)

Slayt: “Aşırı karmaşık if komutlarını sadeleştirmek, anlaşılabilirliği arttırır.”

```
/*** ÖNCE: ***/
if (date.before (SUMMER_START) || date.after(SUMMER_END))
    charge = quantity * _winterRate + _winterServiceCharge;
else charge = quantity * _summerRate;
```

Tarih yaz başlangıcından önce ya da yaz sonundan sonraysa miktar çarpı kış oranı artı sabit bir kış servis bedeli; aksi hâlde miktar çarpı yaz oranı (yazın servis bedeli yok) hesaplanır. Hoca bu kodun doğru olup olmadığını ilk bakışta anlayamadığını, kâğıt kalem oynatmak gerektiğini söyler ve ekranda bir zaman ekseni çizer: eksende `SUMMER_START` ve `SUMMER_END` işaretlenir; yaz başlangıcından önceki ve yaz sonundan sonraki kısımlar yeşille boyanır. Mevsimler döngüsel olduğu için yeşil kısım sonbahar, kış ve ilkbaharı içerir; bunlar kış tarifesine dahil edilmektedir. `else` durumu ise yaz başlangıcı ile bitişi arasındaki kısımdır. Kod doğrudur, ama anlamak için bile çizim gerekmiştir; bu karmaşıktır.

```
/*** SONRA: ***/
if (notSummer(date))
    charge = winterCharge(quantity);
else charge = summerCharge (quantity);

private boolean notSummer(Date date) {
    return date.before (SUMMER_START) ||
    date.after(SUMMER_END);
}
private double summerCharge(int quantity) {
    return quantity * _summerRate;
}
private double winterCharge(int quantity) {
    return quantity * _winterRate + _winterServiceCharge;
}
```

Çizimden anlaşılan iş mantığı, "yaz değilse kış tarifesini, aksi hâlde yaz tarifesini uygula" biçimindedir ve kod bu sözcüklere dökülür. Koşuldaki ifade, verilen tarihin yaz olup olmadığını hesaplayan ayrı bir `notSummer` metoduna çıkarılır; buna "koşulu ayrıştırmak" (decompose conditional) denir. Metot `private` yapılmıştır; aynı sınıfta başka yerden de çağrılabilir. Başka sınıflar da bir tarihin belli bir mevsimde olup olmadığını soruyorsa belki `public static` yapılabilir; buna sistemin geneline bakarak karar verilir.

Hoca karmaşık ifadeyi "halının altına süpürmediğini" belirtir: ifade daha iyi anlaşılır biçimde yazılabiliyorsa öyle yazılmalıdır. Burada doğru olduğundan emin olduktan sonra ifadeyi olduğu gibi bıraktı ama olduğu yerde bırakmadı, ayrı bir metoda taşıdı. Sonradan bir programcı doğruluğunu yeniden denetlemek isterse bütün metodu anlamak yerine yalnız o küçük metoda bakar, hatta yalnız onun testini çalıştırır; doğru çalışıyorsa problemi başka yerde arar.

Toplama ve çarpma işlemleri de ayrı hesaplama sorumlulukları olduğu için `summerCharge` ve `winterCharge` metotlarına çıkarılmıştır. Yalnız koşulu ya da yalnız hesaplardan birini düzeltmek "hibrit" bir çözüm olurdu ve kolay anlaşılır, ayrıntıları iç katmanlarda duran kod hedefine tam ulaştırmazdı.

12. Long Parameter List: Introduce Parameter Object ve Preserve Whole Object

12.1 Kusur

Bir metodun parametre listesi uzunsa ve bu parametrelerin büyük çoğunluğunun başka metotlarda da birlikte geçtiği keşfedilirse bu bir kod kusuru sayılır: Long Parameter List. Bu, söz konusu parametrelerin birlikte bir varlığın durum bilgisinin bir kısmını ya da tamamını oluşturduğunun göstergesidir. İki refactoring eylemi vardır: Introduce Parameter Object ve Preserve Whole Object. Hoca hangisinin ne zaman uygulanacağını bir de "yeni bir varlık olup olmaması" ile ilişkilendirir: parametreler henüz modellenmemiş yeni bir varlığa işaret ediyorsa Introduce Parameter Object, zaten var olan bir nesnenin üyeleriyse Preserve Whole Object uygulanır.

12.2 Introduce Parameter Object (ekrandan, 01:56:25)

Slayt: "Bir metot çok fazla parametre alıyorsa, uygun parametreleri üye alan olarak yeni bir sınıfta toplayın ve o türden bir nesneyi parametre olarak kullanın."

```
ÖNCE:
Customer
  amountInvoicedIn(start: Date, end: Date)
  amountReceivedIn(start: Date, end: Date)
  amountOverdueIn(start: Date, end: Date)

SONRA:
Customer
  amountInvoicedIn(DateRange)
  amountReceivedIn(DateRange)
  amountOverdueIn(DateRange)
```

Aslında iki parametre çok uzun bir liste değildir. Ama aynı parametre çiftinin üç metotta da geçtiği görülür. Demek ki ayrı başlangıç ve bitiş tarihleriyle birçok yerde işlem yapmak yerine sistemde bir zaman aralığı varlığına ihtiyaç vardır; gereksinim belgesinde böyle bir şey söylenmemiş olsa bile. `DateRange` sınıfı oluşturmak parametre sayısını da azaltır, ama hoca buna odaklanmak istemez: kod kusuru açısından düşünülmese bile bu bilgiler birlikte geçtiği için bir varlığın bileşenleridir.

Slayttaki ek maddeler: bunu yalnız parametre listesini kısaltmak amacıyla yapmış olmazsınız; oluşturulan `DateRange` sınıfı yalnız iki veriyi birleştiren bir kayıt yapısı şeklinde kalmaz, `isWithinRange(Date)` gibi uygun metotlar da içererek decompose conditional ya da extract method düzenlemesi de yapılmış olunur. Hoca önceki ders saatinde belirli bir tarihin bir aralığın içinde mi dışında mı olduğunu sınayan bir metot çıkardıklarını hatırlatır; `DateRange` sınıfı varsa o metot oraya konur. İyi tasarımın ilkesi, bir sorumluluğu onu yerine getirecek en fazla veriye sahip varlığa koymaktır. Verilen bir tarihin bir aralığın içinde olup olmadığını en iyi tarih aralığı sınıfı bilir; “taşlar yerine oturur” ve uyum yüksek kalır.

12.3 Preserve Whole Object (ekrandan, 02:00:00)

Slayt: “Bir nesnenin çeşitli üyelerini aynı metodun parametreleri olarak göndermek yerine, nesnenin kendisini gönderin.”

```
/*** ÖNCE: ***/
int low = daysTempRange().getLow();
int high = daysTempRange().getHigh();
withinPlan = plan.withinRange(low, high);

/*** SONRA: ***/
withinPlan = plan.withinRange(daysTempRange());
//withinRange metodu da gerektiği gibi değiştirilir.
```

Örnekte bir günün sıcaklık aralığının en düşük ve en yüksek değerleri geçici değişkenlere alınır ve ısıtma planının aralığında olup olmadığı sorulur. Plan içindeyse normal faturalandırılır; daha yüksek ya da düşük bir değerle karşılaşılırsa daha fazla fatura kesilir. Bu düzenlemeyle geçici değişkenlerden de kurtulunur. Yalnız geçici değişkenlerden kurtulmak amaçlansaydı `low` ve `high` yerine doğrudan `daysTempRange().getLow()` ve `daysTempRange().getHigh()` verilirdi; bu Replace Temp with Query olurdu. Burada ise `daysTempRange()` hangi nesneyi döndürüyorsa o parametre olarak verilir; `withinRange` metodu o nesnenin `low`, `high` ya da başka değerlerini kendisi sorgular.

Slayttaki zayıf yön: bağımlılık artabilir. Parametre olarak gönderilen nesnenin sınıfı ile metodun sahibi olan sınıf arasında bir bağımlılık oluşur; bu durum çok sakıncalı olacaksa bu düzenleme kullanılmaz. Hoca açıklar: değişiklikten önce `plan` nesnesinin sınıfı ile `daysTempRange` metodunun döndürdüğü sınıf arasında hiçbir ilişki olmayabilir. Düzenlemeyle aralarında en azından bir kullanma ilişkisi (kesikli çizgili bir ok daha) kurulur ve bağlaşım yükselir. Bağlaşımı fazla artırmayacaksa çekinmeden yapılmalıdır. Büyük olasılıkla iki sınıf arasında zaten bir sahiplik ya da kullanma ilişkisi vardır; o zaman hoca gözü kapalı yapacağını söyler.

Hocanın slaytta yazmayıp derste eklediği daha büyük sakınca şudur: `withinRange` metodunun imzası değişir. Daha önce iki `int` parametre alırken şimdi başka bir sınıf türünden bir parametre alır. Bu metot başka sınıflar tarafından kullanılıyorsa onlarda da değişiklik gerekir; dışarıya verilen hizmetin arayüzü değişir ve geriye doğru uyumluluk kaybedilebilir. Bu durumda

yapılan iş tam bir refactoring eylemi sayılmaz. Metot başka bir paketten kullanılmıyorsa ya da çok az yerde değişiklik gerekiyorsa sorun yoktur; geriye doğru uyumluluk en büyük öncelikse düzenleme yapılmaz.

13. Replace Temp with Query ve Başarım Tartışması

13.1 Örnek (ekrandan, 02:10:00)

Slayt: “Bir ifadeyi bir geçici değişkende saklayarak kullanmak yerine, ifadeden metot(lar) çıkartın.”

```
/*** ÖNCE: ***/
double getPrice() {
    int basePrice = _quantity * _itemPrice;
    double discountFactor;
    if (basePrice > 1000) discountFactor = 0.95;
    else discountFactor = 0.98;
    return basePrice * discountFactor;
}
```

```
/*** SONRA: ***/
double getPrice() { return basePrice() * discountFactor(); }
private int basePrice() { return _quantity * _itemPrice; }
private double discountFactor() {
    if (basePrice() > 1000) return 0.95;
    else return 0.98;
}
```

İş kuralı şudur: taban fiyat miktar çarpı birim fiyattır. Normalde %2 indirim yapılır; taban fiyat belli bir limitin (örnekte 1000 lira) üstündeyse indirim oranı %5’e çıkar. İndirim çarpanı bu yüzden 0.95 ya da 0.98’dir ve sonuç taban fiyat çarpı indirim çarpanıdır. Önceki kodda taban fiyat da indirim çarpanı da geçici değişkenlerde tutulur. Geçici değişkenler kodu uzattığı için kaldırılır: `getPrice` metodunun asıl yaptığı iş “taban fiyat çarpı indirim çarpanını döndürmek” olduğu için metot o tek satıra iner. Taban fiyat ve indirim çarpanı hesapları ayrı sorgu (query) metotlarına çıkarılır. Sonuçta `getPrice` yaptığı işi ana hatlarıyla özetleyen bir ifade olur; ayrıntılar `basePrice` ve `discountFactor` metotlarındadır. Önceki ve sonraki kod aynı sonucu üretir.

13.2 Başarım bedeli

Sonraki kodda `getPrice` çağrılınca `basePrice` hesaplanır, `discountFactor` çağrılınca `basePrice` bir kez daha hesaplanır. Önceden bir çarpma yapılırken şimdi iki çarpma yapılır. Hoca bunun sorun olup olmadığını sorar ve yanıtlar: artık bir çarpmanın uzun sürdüğü eski ana bilgisayarlarda (mainframe) değiliz; tek bir çarpma işlemi önemsizdir. Metot çağrısının bağlam değiştirme ek yükünü de derleyiciler büyük ölçüde optimize eder; optimize etmese bile sistemin geneline bakınca fark edilmez.

Ama hesaplar aynı bilgisayarın belleğinde çalışan kodlar değilse durum değişir. Örneğin taban fiyat hesabı ayrı bir makinedeki ayrı bir servis, indirim çarpanı hesabı başka bir makinedeki başka bir servisse ve aralarında yüksek gecikmeli (latency) bir ağ ya da servis yolu varsa, bu düzenleme başarımı hissedilir derecede düşürebilir; o zaman yapılmaz. Hoca, dersi alan öğrencilerden bankacılık gibi kurumların bilgi sistemlerinin bu yapıda olma eğiliminde olduğunu öğrendiğini aktarır. Refactoring önemli bir darboğaz oluşturacaksa geri alınır; darboğaz başka yerdeyse yapılmalıdır, çünkü bakım masrafı çok düşer.

Bu düzenlemenin kazancı: geçici değişkenler hem kodu uzatmaya meylettirir hem karmaşıklığı artırır hem de kopyala-yapıştır kodlamayı teşvik eder. Bir sınıfın bir metodunda geçici değişkenle daha az işlem yaparak kodlanmış bir hesaplamaya başka yerde de ihtiyaç duyulunca “az önce orada yapmıştım, kopyalayayım” diye düşünülür; bu da yinelenen kodun sakıncalarına yol açar.

13.3 Döngü içinde geçici değişken (ekrandan, 02:14:40)

Slayttaki tartışma maddeleri: geçici değişken metot içinde birden fazla yerde kullanılıyorsa aynı işlem tekrar tekrar yapılacaktır; bu da başarımı daha da fazla düşürecektir.

```
void printRecipt() { /*** ÖNCE: ***/
    int totalAmount = 0, thisAmount;
    for( Item item : items ) {
        thisAmount = item.getCharge( );
        System.out.println(item.getName()+" "+thisAmount);
        totalAmount += thisAmount;
} }
```

```
void printRecipt() { /*** SONRA: ***/
    int totalAmount = 0;
    for( Item item : items ) {
        System.out.println(item.getName()+" "+item.getCharge());
        totalAmount += item.getCharge( );
} }
```

İki geçici değişken vardır. `totalAmount` kaldırılamaz, çünkü problem zaten kümülatif toplamı bulmaktır ve bir yerde saklanması gerekir. `thisAmount` ise bir kez atanır ve sonra hiç değiştirilmeden yalnız kullanılır; IDE'de üzerine tıklanınca bütün kullanımları vurgulanır. Bu yüzden `thisAmount` kaldırılıp geçtiği her yere `item.getCharge()` yazılırsa refactoring tamamlanır.

Ancak bu çağrı bir `for` döngüsünün içindedir: önceden öğe sayısı kadar çağrılırken şimdi öğe sayısının iki katı kadar çağrılır. On ya da yüz öğede fark etmez, ama beş bin, on bin, milyonlarca öğede fark edebilir. Özellikle `getCharge` hesapları uzun sürüyorsa, örneğin başka bilgi sistemlerinden karmaşık sorgularla veri çekiliyorsa, bu düzenleme yapılmaz ve kod eski hâlinde kalır. Böyle bir durum yoksa geçici değişken çekinmeden kaldırılır. Önce böyle bir durum olup olmadığına bakılır; birim testleri çalıştırılırken süreleri de raporlanır, yalnız koşma süresini ölçmek için ayrı bir test durumu da hazırlanabilir. Örneğin toplamda %5'lik hissedilir bir başarım düşüşü önemli bir rakam olabilir; o zaman bu refactoring yapılmaz ve daha az başarım yüküyle daha fazla bakım yararı sağlayacak yerler refactor edilir.

14. Replace Method with Method Object

14.1 Ne zaman? (ekrandan, 02:20:00)

Slayt maddeleri:

- Metot öyle uzun ve geçici yerel değişkenleri öyle "arap saçı" gibi kullanıyor ki extract method düzenlemesini kullanamıyorsunuz.
- O zaman yeni bir sınıf oluşturup bütün metodu o sınıfa taşıyın, yerel değişkenleri de o sınıfın üyeleri yapın.
- Ardından bildiğiniz düzenleme eylemlerini daha rahat yürütebilirsiniz.
- Yeni sınıfın adını verirken taşınan metodun adına benzer bir ad verin.

Hoca durumu şöyle tarif eder: çok karmaşık bir uzun metot çok sayıda geçici yerel değişken kullanır, bu değişkenlerin değerini değiştirip aynı metodun başka yerlerinde değişmiş hâlini kullanır; iki üç döngü, ayrıntılı ve karmaşık if koşulları vardır. Extract Method için basit bir çözüm bulunamaz. O zaman en azından bütün o karmaşıklık yumağı bulunduğu sınıftan çıkarılır; o sınıfın kodu temizlenir ve karmaşıklık kendi içinde barındırılmak üzere yeni bir sınıfa taşınır. Yalnız o sınıfa odaklanınca küçük değişikliklerle kodun nasıl sadeleştirileceği belki görülebilir; ilk aşamada kalabalıktan görülememiş olabilir. Taşıdıktan sonra bile sadeleştirilemiyorsa öyle kalır, ama en azından asıl sınıf karmaşıklıktan kurtulmuştur.

14.2 Örnek

```
ÖNCE:
class Order {
   double price() {
   double primaryBasePrice;
```

```
   double secondaryBasePrice;
   double tertiaryBasePrice;
   // long computation;
   ...
}

SONRA:
Order {price()}  - - - kullanır - - ->  PriceCalculator
PriceCalculator {primaryBasePrice; secondaryBasePrice; tertiaryBasePrice; compute()}
PriceCalculator ──1──► Order
not: price() içinde  return new PriceCalculator(this).compute()
```

`Order` sınıfının `price` metodunda üç geçici değişken ve çok uzun hesaplamalar vardır; `primaryBasePrice` bir yerde atanır, bir if-else içinde başka şeylerle çarpılıp değiştirilir ve dallar bittikten sonra da kullanılır. Bu metot bütünüyle çıkarılır ve `PriceCalculator` adlı yeni bir sınıfa konur. Yapılan iş artık Extract Method değil, yeni bir sınıf oluşturmaktır (bir bakıma Extract Class). Ama sınıf güzellik olsun diye değil, karmaşıklığı barındırsın diye oluşturulmuştur; bu eylemlerin bütününe Replace Method with Method Object denir. Kesilen metodun geçici değişkenleri yeni sınıfın üye alanları olur; işlem, aynı adı taşıyan ya da daha anlaşılır bir ad verilen bir metoda (`compute`) alınır.

`Order.price` içinde yeni bir `PriceCalculator` nesnesi oluşturulup `compute` çağrılır (metot statik olsaydı doğrudan çağrılabilirdi, ama büyük olasılıkla değildir). Duruma göre `PriceCalculator` bir Singleton da olabilir. `price` metodu artık hesaplamayı başkasına yaptıran bir delegate metot olmuştur; asıl iş yeni sınıfta yapılır. `PriceCalculator` sınıfından türeyen nesneler yeni "metot nesneleri"dir.

15. Large Class: Extract Class, Extract Subclass, Duplicate Observed Data

15.1 Kusur

Large Class, sınıfın kendisinin büyük olmasıdır. Bir sınıf ne zaman büyük sayılır? Bunun tek bir rakamsal yanıtı yoktur. Bir görselleştirme aracıyla yazılımın tümünün resmi çekilirse en büyük sınıflar kolayca görülür. Böyle bir araç yoksa bile `.java` dosyalarının satır sayılarına ya da derlenen `.class` dosyalarının boyutlarına bakılabilir: bütün sınıflar 200-300 KB iken birinin 3 MB olması, yani diğerlerinden on kat büyük olması dikkat çekicidir. Üye alan sayısı da sinyaldir (diğer sınıflarda 8-10, birkaçında 30); metot sayısı da (genelde 20, birinde 50-60). Sonar gibi kod analiz yazılımları ortalamadan çok büyük sınıfları ve her sınıfın bağlaşım ve uyum metriklerini hızlıca gösterir; Eclipse için basit bir eklenti bile vardır.

Böyle bir sınıf büyük olasılıkla çok fazla iş yapmaya çalışır; ona çok fazla sorumluluk atanmıştır ve bu sorumluluklar arasında birbiriyle ilişkisiz olanlar vardır. Bu, sınıfın uyumunun çok düşük olduğu anlamına gelir. Bu sınıflar bölünmeli, refactor edilmelidir. Seçenekler: yeni bir sınıf oluşturmak (Extract Class), bir kalıtım hiyerarşisi kurmak (Extract Subclass) ya da Observer tasarım kalıbının bir gerçeklemesine dayanan, nispeten karışık bir düzenleme olan Duplicate Observed Data.

15.2 Extract Class (ekrandan, 02:28:00)

Slayt maddeleri: birbiri ile ilgisiz iki iş yapan bir sınıfı bu işlere göre ikiye ayırmaktır; ikinci iş ile ilgili üyelerin seçilerek yeni bir sınıfa taşınmasına dayanır; bu yapılandırma da birçok kod kusurunu düzeltir.

```
ÖNCE:
Person {name; officeAreaCode; officeNumber; getTelephoneNumber}

SONRA:
Person {name; getTelephoneNumber}  ──officeTelephone, 1──►  TelephoneNumber {areaCode; number;
```

`Person` sınıfında ad, ofis telefonunun alan kodu ve numarası vardır; `getTelephoneNumber` bunları birleştirip `String` olarak döndürür. Bir insan varlığının içinde bir telefon numarası varlığı

gizlenmiştir. Bu yüzden `TelephoneNumber` sınıfı çıkarılır; alan kodu ve numara oraya taşınır. `getTelephoneNumber` metodu ise kopyalanır, çünkü `Person`'dan kesilemez; imzası kopyalanır, gövdeleri farklı olur. Aralarına birebir sahiplik ilişkisi kurulur. Hocaya göre aslında baştan böyle yapılmalıydı: iyi bir tasarım, problem alanı hangi düzeyde varlıklara ayrılıyorsa o düzeyde ya da daha ayrıntılı ayrılan tasarımdır; kişinin telefonu ayrı bir varlık ve ayrı bir bilgidir. Ayrılan sınıf, iki telefon numarasının aynı bölgede ya da ülkede olup olmadığı, bir çağrının uluslararası olup olmayacağı gibi işlevler için de iyi bir yer oluşturur.

15.3 Extract Subclass (ekrandan, 02:30:00, 02:40:00)

Slayt: "Bir sınıfın bazı üyeleri her zaman kullanılmıyorsa, bu üyelerden yeni bir alt sınıf oluşturmak daha doğru olacaktır."

```
ÖNCE:
JobItem {getTotalPrice; getUnitPrice; getEmployee}

SONRA:
JobItem {getTotalPrice; getUnitPrice}
JobItem ◁── LaborItem {getUnitPrice; getEmployee}
```

`JobItem` sınıfında toplam fiyat, birim fiyat ve işin sorumlusu olan çalışan vardır. Bir iş genellikle herhangi bir çalışana yaptırılabilir, ama bazı daha özelleşmiş işler belirli, uzman bir çalışana yaptırılmak zorundadır; o durumda birim fiyat hesaplanırken uzman çalışanın emeği için daha fazla masraf çıkabilir. Bu bir kalıtım ilişkisini ortaya çıkarır: `JobItem`'in yanında daha özelleşmiş bir `LaborItem` vardır. Özelleşmiş kısımla ilgili kodlar `LaborItem`'a taşınır; uzun ve karmaşık `JobItem` sınıfı daha odaklanmış, kendi içinde uyumu yüksek ve daha az karmaşık iki sınıfa ayrılır. Hoca ekranda `JobItem`'in sonraki hâlinde yalnız `getUnitPrice`'ın kaldığını, `getTotalPrice`'ın çizildiğini işaretler.

15.4 Duplicate Observed Data (ekrandan, 02:34:30)

Slayt maddeleri: uygulama alanı ile ilgili bir verinin yalnız bir GUI sınıfında saklanması doğru değildir; bu veri uygulama alanı/bilgi düzeyi katmanında oluşturulacak yeni bir sınıfa taşınmalı ve GUI nesnesi ile yeni bilgi nesnesi arasında Observer tasarım kalıbı kurulmalıdır.

```
ÖNCE:
IntervalWindow {startField: TextField; endField: TextField; lengthField: TextField;
                StartField_FocusLost; EndField_FocusLost; LengthField_FocusLost;
                calculateLength; calculateEnd}

SONRA:
IntervalWindow {startField: TextField; endField: TextField; lengthField: TextField;
                StartField_FocusLost; EndField_FocusLost; LengthField_FocusLost}
IntervalWindow - - -▷ «interface» Observer
IntervalWindow ──1──► Interval
Interval {start: String; end: String; length: String; calculateLength; calculateEnd}
Interval ──▷ Observable
```

Arka planda bir ilişkisel veritabanına doğrudan bağlı akıllı GUI bileşenleri ya da hazır bir çerçeve kullanılıyorsa bu karmaşıklığı o bileşenler halleder ve müdahale gerekmez. Ama her şey sıfırdan kodlanıyorsa ya da günü kurtarmak için, asıl amacı yalnız kullanıcıdan bilgi almak olan GUI bileşenini simgeleyen sınıfa (`IntervalWindow`) iş mantığı kodu yazma eğilimi olabilir. Bu durumda sunum katmanı ile iş mantığı katmanına birden bulaşmış bir sınıf elde edilir ve katmanlı mimari bozulur.

Çözümde veri çoğaltılır: `IntervalWindow` kalır, ama iş mantığı katmanında bir `Interval` nesnesi de olur ve pencere bu nesneyi üye alan olarak tutar. GUI olayları gerçekleştikçe `Interval`'ın set metotları çağrılıp nesne güncellenir; asıl işlemleri `Interval` yapar. `Interval`'ın durumu başka bir nedenle değişebilir; örneğin çok kullanıcılı bir sistemde başka bir kullanıcı aralık bilgisini değiştirmiş olabilir. O zaman değişikliğin GUI penceresine aktarılması gerekir. Bu yüzden Observer kalıbı kurulur: GUI penceresi Observer, `Interval` Observable olur; tersi geçerli değildir.

Bu çözüm daha masraflıdır: veri çoğaltılır ve bir tasarım kalıbı kurmak gerekir, basit bir refactoring değildir. Ama önceki hâl katmanlı mimariyi bozuyordu; bu düzenleme saf bir katmanlı mimariye dönmeyi sağlar. Hoca bu yüzden mutlaka tercih ettiğini söyler.

16. Divergent Change ve Shotgun Surgery

Hoca birbiriyle ilişkili bu iki kusuru ortak bir kök nedende toplar: değişikliklerin farklı noktalarda yapılmasının gerekmesi. Bir hata bulunur ya da gereksinim değişir ve kodda bir yerler değiştirilmelidir. İyi bir tasarım yapılmışsa ve analiz dikkatle yürütülmüşse düzeltmeler en fazla bir iki sınıfın iki üç metodunda olur. Aksi hâlde on beş, yirmi yerde değişiklik gerekir; bu, tasarımın baştan doğru yapılmadığını gösterir. Değişikliklerin nerede toplandığına göre kusurun adı değişir:

- **Divergent Change:** değişiklikler aynı sınıfın çok sayıda metodunda yapılıyorsa. Örneğin bir sınıfın üç farklı metodunda toplam dört değişiklik.
- **Shotgun Surgery:** değişiklikler çok sayıda sınıfa yayılmışsa; bir sınıfın iki metodu, öbür sınıfın üç metodu, bir başkasının bir metodu gibi.

Shotgun Surgery adının akılda kalması için hoca şu benzetmeyi yapar: saçmalı tüfek (çifte) küçük küçük saçmalarla ateş eder ve bunlar hedefin etrafına bulut gibi yayılır. Normal mermiyle kaç kez ateş edildiyse o kadar delik olur; saçmalı tüfekte tetiği bir kez çekmek onlarca delik açar. Durum, çifteyle ameliyat etmeye çalışmaya benzer. Hoca bunu ekranda bir sınıf şeması üzerinde çizerek gösterir.

İki kusurun adı farklıdır ama çözüm araçları büyük oranda ortaktır: Extract Class (belki Extract Subclass), ya da Move Method ve Move Field. Değişiklik yapılması gereken metot ya da alanlar yeni oluşturulmayacak, zaten var olan daha uygun bir sınıfa taşınacaksa Move Method veya Move Field yapılır.

16.1 Move Method

Bir metot bir sınıftan alınıp başka bir sınıfa taşınır. Taşıma sırasında metodun imzası ilk yerinde de kalabilir; o zaman eski yerdeki metot yalnız tek satırdan oluşur ve çağrıyı yeni yere iletir, asıl gövde yeni sınıftadır. Bu, iki sınıf arasında bir ilişki varsa olabilir; yoksa tamamen bir taşıma olur. İki durum da Move Method sayılır; arada delegate ilişkisi kurulmuş olsun olmasın fark etmez.

Neden taşınır? Örneğin bir metot kendi sınıfında yalnız iki farklı metottan çağrılırken diğer sınıfın 8-10 ayrı metodundan çağrılıyordur. O zaman bu metoda asıl ihtiyacı olan diğer sınıftır ve oraya konmalıdır. Move Method birçok başka kod kusurunu da düzeltebilir. Basit ama işe yarayan bir fikirdir ve daha karmaşık refactoring eylemlerinin alt adımı da olabilir: örneğin Extract Class yaptıktan sonra bazı metotların yeni sınıfa taşınmasının daha anlamlı olduğu fark edilir.

17. Primitive Obsession ve Tür Kodu Düzenlemeleri

17.1 Kusur ve kökeni

Primitive Obsession birebir çevrilirse “ilkel türlere takıntı”dır: ilkel tür sınıf olmayan `int`, `long`, `char` gibi türlerdir. Hoca “takıntı” yerine “aşırı kullanım” demeyi tercih eder, ama özgün adın bilinmesi gerekir.

Birçok kod kusurunun nedeni, başka bir programlama yaklaşımını önce öğrenip uzun süre kullanmış olmamızdır. Örneğin yıllarca yapısal programlama dilleriyle çalışıp sonra nesneye yönelik kod yazmaya geçen biri için öğrenmedeki “geriye ket vurma” kavramı geçerlidir: daha önce edinilen bilgi yeni bilgilerin kullanılmasının ya da edinilmesinin önüne geçer. Nesneye yönelim yeni öğrenilmiştir, iyi uygulamalarına (best practices) henüz vakıf olunmamıştır ve yapısal programlamadan gelen alışkanlıklar sürdürülür. Geçici değişkenleri aşırı kullanmak ve uzun fonksiyonlar yazmak bunlara örnektir. Primitive Obsession’ın kökü tamamen budur.

Bu aşırı kullanımın farklı çeşitleri vardır ve her biri için farklı düzenlemeler gerekir: Replace Data Value with Object, Replace Type Code with Class, Replace Type Code with Subclass, Replace Type Code with State/Strategy, Replace Array with Object. Bunlardan önce Extract Class

ve Introduce Parameter Object de kullanılabiliyorsa onlar da Primitive Obsession'ı düzeltebilir; hangisi daha doğruysa o seçilir.

17.2 Replace Data Value with Object (ekrandan, 02:50:00)

Slayt: "Bir üye alan, hele ki bu alan ile ilgili çeşitli davranışlar söz konusu ise, bir sınıf ile simgelenmelidir."

```
ÖNCE:
Order {customer: String}

SONRA:
Order ◆──1──► Customer {name: String}
```

`Order` sınıfında siparişi veren müşteri `String` olarak tutulur. Hoca kendi adını örnek verir: büyük/küçük harf duyarlılığı vardır; ad yalnız ilk harfleri büyük, tamamen büyük harfle ya da iki ad ve iki soyadla (örneğin kızlık soyadını koruyan biri için) farklı biçimlerde girilebilir. Sonra bu müşterinin adresi gibi bilgilere ulaşmak için müşteri listesinde o ada sahip müşteriler aranır; bir tane çıkarsa sorun yoktur, iki üç tane çıkarsa hangisi olduğu bilinemez. Müşteriye basit bir `String` gibi davranmak yerine onun ayrı bir sınıfı olmalı ve ad bilgisi o sınıfın üye alanı olmalıdır; doğru modelleme budur. Refactor edilmemiş hâl büyük olasılıkla iyi bir analiz yapılmadığını gösterir. Belki ilk gereksinimlerde müşterilerin ayrı ayrı izlenmesi gerekmiyordu; sonra başka varlıkların da bir müşteri nesnesiyle ilişkili olması gerektiği fark edilince müşteri başlı başına bir varlık olur. Yeni `Customer` sınıfı ayırt edici bilginin dışında başka üye alanlara ve metotlara da sahip olur.

Slayttaki ek maddeler: programın tümü incelendiğinde belli bir müşterinin tüm siparişlerinin aranması gibi işlemler yapıldığı görülecektir; bu işlemler çoğaldıkça müşteri sınıfı ayrılmadığı sürece sipariş sınıfı aşırı büyüyecektir; iyisi mi şimdiden bu sınıfları ayıralım.

17.3 Replace Type Code with Class

Bu kusur da büyük olasılıkla yapısal programlamadaki sabit değişken kullanma alışkanlığından kaynaklanır. Örnekte kişinin bilgileri arasında kan grubu da tutulmak istenir. Kan grubu yalnız insanlarla ilgili kabul edilirse, insan sınıfına `public final static` sabitler olarak kan grubu türleri tanımlanır: O için 0, A için 1, B için 2, AB için 3. Hoca AB'ye 3 vermesini ikilik gösterimle açıklar: A `01`, B `10` ise AB `11`, yani 3 olur. Her insan nesnesi için normal bir `bloodGroup` üyesi tanımlanır ve ona bu sabitlerden biri ya da doğrudan 0-3 arası bir sayı atanır.

Oysa kan grubu kişinin ayrı bir özelliğidir; ilgili kod ayrı bir `BloodGroup` sınıfına taşınmalıdır. Kullanılan dil enum yapısını destekliyorsa en doğru çözüm zaten enum kullanmaktır. Enum olmayan bir dille çalışılıyorsa tür koduyla ilgili kodları içerecek ayrı bir sınıf oluşturulur. Eski (legacy) bir sağlık bilgi sistemi gibi bu kod üzerine çok işlev kurulmuş bir kodla uyumluluk sağlamak gerekiyorsa enum yerine tür kodu sınıfla modellenir; aksi hâlde enum kullanılır.

17.4 Replace Type Code with Subclasses (ekrandan, 02:57:20)

Bir başka yanlış kullanım: sistemdeki çalışanlar mühendis, satıcı ya da yönetici olabilir. Kan grubuna benzer biçimde tür kodları tanımlanır ve her çalışanın türü normal bir üyede tutulur. Sonra çalışana bir iş yaptırılacağı zaman sınıf kodunun her yerinde "türü mühendisse şunu yap, değilse ve satıcıysa bunu yap" biçiminde if-else zincirleri ya da switch-case blokları görülür.

```
/*** ÖNCE: ***/
class Employee {
  private int _type;
  static final int ENGINEER = 0;
  static final int SALESMAN = 1;
  static final int MANAGER = 2;
  Employee (int type) {
    _type = type; }
  public void work( ) {
    switch(_type) {
```

```
      case ENGINEER:
        //do something
        break;
      case SALESMAN:
        //do something
        break;
      case MANAGER:
        //do something
        break;
    }
  }
}

/*** SONRA: ***/
abstract class Employee {
  public abstract void work(){};
}
public class Engineer extends
   Employe {
  public void work( ) {
    //do something as an engineer
  }
}
public class Salesman...
```

> **Ders sonrası not:** Slayttaki `public abstract void work(){};` satırı Java'da derlenmez, çünkü soyut bir metodun gövdesi olamaz; doğrusu `public abstract void work();` biçimindedir. Aynı slayttaki `extends Employe` de `extends Employee` olmalıdır. Hoca bu iki noktaya değinmedi.

Hoca, dersin başından beri söylenen ilkeyi hatırlatır: kendinizi bir if-else zinciri ya da switch-case içinde bulursanız bu, çok biçimliliği kullanmanız gerektiğinin işaretidir. Bu, tasarım kalıpları açısından söylenmişti; refactoring açısından bakınca da böyle kodlar üç tür için üç katı uzun metotlar demektir. Tür sayısı dört beş olabilir ve yeni bir çalışan türü eklemek birçok yerde değişiklik gerektirir; bu bir tür Divergent Change, başka sınıflar da bu kodla çalışıyorsa Shotgun Surgery olur. Bu yüzden tür kodu bir kalıtım hiyerarşisiyle, yani alt sınıflarla modellenir: Replace Type Code with Subclass. Farklı çalışan türleri arasında bir özelleşme-genelleşme ilişkisi olduğu için bu daha doğrudur. Soyut bir `Employee` sınıfının soyut bir `work` metodu olur ve her somut alt sınıf bu işte ne yapılması gerekiyorsa onu yapar. Toplam satır sayısı belki biraz artar, ama kod çok daha odaklanmış ve iyi anlaşılır olur; her nesne kendine özel işi yapar.

Slayttaki notlar:

- Employee'nin önceki hâlinde `case default` varsa sonraki hâlde `Employee` sınıfı `abstract` yapılmaz. Varsayılan durum normal `Employee` sınıfının `work` metoduna konur, alt sınıflar onu kendi özelleşmiş hâlleriyle `override` eder.
- Bu örnek Replace Conditional with Polymorphism düzenlemesine de benzemektedir. Hoca bu düzenlemenin kitapta tanımlı olduğunu ama notlarına almadığını söyler.

17.5 Replace Type Code with State/Strategy (ekrandan, 03:00:00)

Slayt: "Önceki örnekte kalıtım kullanımı bir nedenle mümkün değilse, State veya Strategy tasarım kalıbı da kullanılabilir."

```
ÖNCE:
Employee {ENGINEER: int; SALESMAN: int; type: int}

SONRA:
Employee ──1──► EmployeeType
EmployeeType ◁── Engineer
EmployeeType ◁── Salesman
```

Problemin durumuna ve ayrıntısına göre State ya da Strategy kalıbı daha doğru olabilir. Hoca,

bir öğrencinin sorusundan sonra bu örnek için şahsen Replace Type Code with Subclass'ı tercih ettiğini, burada State ya da Strategy kurmaya gerek olmadığını söyler. Ama önemli bir istisna vardır.

Soru (hocanın kendi sorusu): Bir çalışan zaman içinde türünü değiştirebiliyor mu? **Cevap:** Birçok firmada bu görülür. Firma hazır yazılım ürünleri (off-the-shelf software) geliştirip pazarlıyorsa, satış işlerine ilk görev olarak bir bilgisayar mühendisi (belki yeni başlayan ya da MBA yapmış biri) atanabilir; bir mühendis yöneticiye ya da satıcı yöneticiye terfi edebilir; performansı beğenilmeyen bir teknik yöneticinin yetkisi alınıp yeniden mühendis yapılabilir. Kalıtımlı çözümde bir mühendisi yönetici yapmak için mühendisin değişmeyen ortak bilgileriyle yeni bir yönetici nesnesi oluşturmak, yöneticiliğe özgü ek bilgileri ayarlamak ve mühendis nesnesini silmek gerekir; bu uzun bir iştir. Bir varlık kardeş sınıflar arasında türünü değiştirebiliyorsa ve bu belli bir sıklığın üstünde oluyorsa Strategy kalıbı daha doğru bir çözümdür: çalışana "mühendis" stratejisi verilir ve her işi, bonusu, primi, ikramiyesi ona göre hesaplanır; satış görevine atanınca stratejisi değiştirilir. Türler iyi tanımlıysa ve bir nesne türler arasında dolaşmayacaksa alt sınıflı ilk çözüm yeterlidir.

17.6 Replace Array with Object

Primitive Obsession'ın son çeşidi, kayıtları çok boyutlu dizilerde tutmaktır. Örneğin bir lig puan tablosu oluşturulurken iki boyutlu bir matris kullanılır; her satırda takımın adı, puanı ve averajı tutulur ve matris olsun diye sayılar bile `String` olarak saklanır. Ya da üç ayrı dizi olur: takımlar, puanlar ve averajlar; her üç dizinin aynı indisindeki bilgiler aslında aynı varlığın bilgileridir. Hocaya göre nesneye yönelik programlama yapılıyorsa böyle yapılmamalıdır: adı, güncel puanı ve averajı üye alan olan bir `Takım` (ya da hangi takıma ait olduğunu, puanı ve averajı tutan bir `Skor`) sınıfı tanımlanır ve kayıtlar bu sınıfın nesneleriyle tutulur. Çok boyutlu dizilerde kayıt tutmak yerine tek bir nesne türü tanımlanır.

Örnek Sorular

Soru 1 (derste sözlü, 00:11:47)

Font seçme penceresinde liste, metin kutusu, kalınlık, italik ve punto bileşenlerindeki her değişiklik önizleme alanına yansıtılmalıdır. Bu problem için hangi kalıp kullanılır ve bu kalıp Observer'dan nasıl ayrılır?

> **Cevap (derste verildi):** Mediator kalıbı; bu örneğe doğrudan hitap eden kalıptır. Bileşenler (`Widget` alt sınıfları) değiştiklerini `Changed()` içinde `director->WidgetChanged(this)` ile `FontDialogDirector`'e bildirir; director bileşenleri sorgulayıp önizlemeyi ve diğer bileşenleri günceller. Şema Observer'a benzese de amaç değişiklikten haberdar etmek değil, eşten eşe kurulacak $m \times n$ ilişkiyi tek bir arabulucu nesnede toplamaktır.

Soru 2 (ekrandan, 01:40:00)

Aşağıdaki kodu Decompose Conditional ile düzenleyin:

```
if (date.before (SUMMER_START) || date.after(SUMMER_END))
    charge = quantity * _winterRate + _winterServiceCharge;
else charge = quantity * _summerRate;
```

> **Cevap (derste verildi):** Koşul `private boolean notSummer(Date date)` metoduna, iki hesap `winterCharge(int quantity)` ve `summerCharge(int quantity)` metotlarına çıkarılır; ana kod `if (notSummer(date)) charge = winterCharge(quantity); else charge = summerCharge(quantity);` olur. Koşul "yaz başlangıcından önce ya da yaz sonundan sonra", yani "yaz değil" anlamına gelir.

Soru 3 (derste sözlü, 02:09:29)

Replace Temp with Query sonrasında `getPrice()` çağrılınca `basePrice()` iki kez hesaplanır. Bu bir sakınca mıdır?

> **Cevap (derste verildi):** Aynı makinede çalışan kod için hayır: fazladan bir çarpma ya da metot çağrısının ek yükü hissedilmez ve derleyiciler bunu büyük ölçüde optimize eder. Ama hesaplar yüksek gecikmeli bir ağ üzerinden ayrı makinelerdeki servislerle yapılıyorsa ya da çağrı büyük bir döngü içindeyse ve başarım hissedilir biçimde düşüyorsa düzenleme yapılmaz. Darboğaz başka yerdeyse düzenleme yapılmalıdır. Önceki ve sonraki kodun aynı sonucu verdiği, ajan tarafından Python ile çeşitli miktar ve fiyat değerleri için denetlendi.

Soru 4 (derste sözlü, 02:22:55)

Bir sınıf ne zaman Large Class kusuru sayılır?

> **Cevap (derste verildi):** Tek bir rakamsal eşik yoktur. Sınıf, sistemdeki diğer sınıflara göre belirgin biçimde büyükse kusurlu sayılır: dosya ya da `.class` boyutu (diğerleri 200-300 KB iken 3 MB), üye alan sayısı (diğerlerinde 8-10, onda 30), metot sayısı (genelde 20, onda 50-60). Görselleştirme ve kod analiz araçları (ör. Sonar) bunu kolayca gösterir. Böyle bir sınıf büyük olasılıkla birbiriyle ilgisiz çok sayıda sorumluluk taşır ve uyumu düşüktür; Extract Class, Extract Subclass ya da Duplicate Observed Data ile bölünür.

Soru 5 (derste sözlü, 03:01:00)

Çalışan türleri için Replace Type Code with Subclass mı, yoksa Replace Type Code with State/Strategy mi seçilmelidir?

> **Cevap (derste verildi):** Türler iyi tanımlıysa ve bir çalışan türler arasında dolaşmayacaksa alt sınıflı çözüm yeterlidir; hoca bu örnekte onu tercih eder. Ama bir çalışan zamanla mühendisten yöneticiye terfi edebiliyor, satış görevine geçebiliyor ya da yöneticilikten geri alınabiliyorsa ve bu sık oluyorsa, her seferinde nesneyi yeniden oluşturup eskisini silmek yerine çalışana bir `EmployeeType` stratejisi atamak daha doğrudur.

Dersin Toparlaması

Bu derste tasarım kalıpları konusu Mediator ile kapandı. Mediator, eşten eşe haberleşen nesneler arasındaki karmaşık ilişkiyi tek bir arabulucu nesnede toplar. Font penceresi örneğinde bileşenler değişikliklerini `FontDialogDirector`'e bildirir, o da diğer bileşenleri ve önizlemeyi günceller. Şeması Observer'a benzese de amacı farklıdır. Karmaşıklığı yok etmez, merkeze taşır; bu yüzden ancak etkileşimler iyi tanımlı ve kararlıysa kurulmaya değer.

Dersin büyük kısmı refactoring'e ayrıldı. Refactoring, dış davranışı değiştirmeden iç yapıyı iyileştirmektir; küçük adımlarla ve her adımdan sonra birim testlerle yapılır. Bu yüzden JUnit 3.x (`TestCase`, `setUp`, `test` önekli metotlar, `suite`, `assertXxx`) ve 4.x (`@Test`, `@Before`, `@Test(expected=...)`) tanıtıldı ve Fowler'ın video kiralama örneğinin ilk hâli testleriyle gösterildi. Ardından kod kusurları ve onları gideren düzenlemeler sırayla işlendi: Duplicated Code ve Long Method için Extract Method, Pull Up Method, Form Template Method, Decompose Conditional, Replace Temp with Query, Replace Method with Method Object; Long Parameter List için Introduce Parameter Object ve Preserve Whole Object; Large Class için Extract Class, Extract Subclass ve Duplicate Observed Data; Divergent Change ve Shotgun Surgery için Move Method ve Move Field; Primitive Obsession için Replace Data Value with Object, Replace Type Code with Class/Subclass/State-Strategy ve Replace Array with Object. Bütün tartışmaların ortak ekseni, bir düzenlemenin getirdiği bakım yararını başarım ve bağlaşım bedeline karşı tartmaktı. Hoca bir kod kusurunun gelecek derse kaldığını ve ardından Fowler'ın kitabındaki bütüncül refactoring örneğinin adım adım yapılacağını söyledi.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.